

Comunicação Síncrona e Assíncrona em MPI: Análise de Desempenho na Multiplicação de Matrizes

Rayan Raddatz de Matos, Enzo Lisboa Peixoto, Eduardo Magnus Lazuta

Introdução

Este trabalho compara três formas de comunicação em MPI aplicadas à multiplicação de matrizes:

1. **Coletiva:** usa `MPI_Scatter`, `MPI_Bcast` e `MPI_Gather`, deixando a biblioteca decidir a estratégia de comunicação internamente.
2. **Ponto a ponto bloqueante:** o mestre envia os dados com `MPI_Send` e os trabalhadores recebem com `MPI_Recv`; cada chamada bloqueia até a transferência terminar. A matriz B ainda é distribuída com `MPI_Bcast`.
3. **Ponto a ponto não bloqueante:** usa `MPI_Isend`, `MPI_Irecv` e `MPI_Ibcast`, que retornam imediatamente. A sincronização é feita depois com `MPI_Wait`.

O algoritmo divide as linhas da matriz A entre os processos, replica B em todos eles, cada processo calcula sua parte de C e devolve o resultado ao mestre.

Metodologia

Os experimentos rodaram na partição **hype** do cluster PCAD (nós com 2x Xeon E5-2650 v3, 20 núcleos físicos por nó), com MPI sobre TCP e sem fixação de processos a núcleos (`--bind-to none`). Todas as combinações dos fatores abaixo foram executadas 2 vezes (324 execuções no total):

Fator	Níveis
Tipo comunicação	coletiva, p2p bloqueante, p2p não bloqueante
Nós	1, 2, 3
Processos por nó	1, 2, 4, 8, 16, 32
Tamanho (NxN)	500, 1000, 1500

Cada execução foi rastreada com `akypuera`: toda chamada MPI de todo processo fica registrada com início, fim e duração. As métricas usadas são:

- **Tempo total:** medido com `MPI_Wtime` no processo 0, do início da distribuição até o fim da coleta.
- **Tempo por função MPI:** soma de quanto cada processo passou dentro de cada função, tirando a média entre os processos. Responde "em média, quanto um processo gasta nessa função?"
- Como cada configuração rodou 2 vezes, reportamos a mediana e usamos o desvio padrão como indicação de variabilidade.

`MPI_Finalize` é excluído da análise por função: sua duração é apenas espera pelos processos mais lentos, não comunicação.

Linha do tempo das execuções (Gantt)

A forma mais direta de comparar as versões é olhar a linha do tempo de uma execução: cada linha horizontal é um processo e o tempo corre para a direita. Trechos coloridos são chamadas MPI; trechos em branco são computação;

o cinza no final é o `MPI_Finalize`.

Usamos a maior configuração do experimento, 1500×1500 , 96 processos em 3 nós (32 por nó), porque é onde a comunicação mais aparece. Os processos 0–31 estão no mesmo nó do mestre; os demais estão em nós remotos.

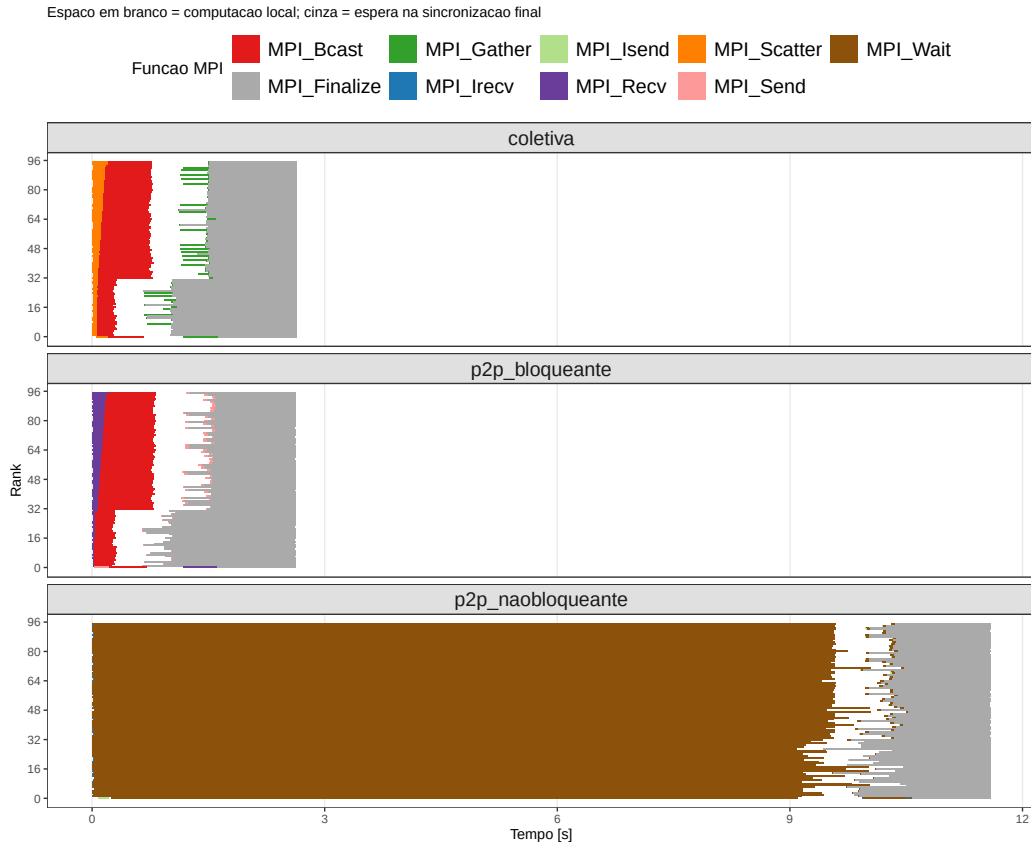


Figura 1: Uma execução de cada versão (size=1500, 96 processos, 3 nós). Cada linha horizontal é um processo.

Nas versões coletiva e bloqueante a comunicação ocupa só o começo e o fim, e tudo termina em ~2.7s. Na não bloqueante quase todo o gráfico é marrom (`MPI_Wait`): os processos passam a maior parte dos ~10s esperando dados chegarem.

Para ver as diferenças de semântica entre as versões, ampliamos os primeiros 0.6s (fase de distribuição):

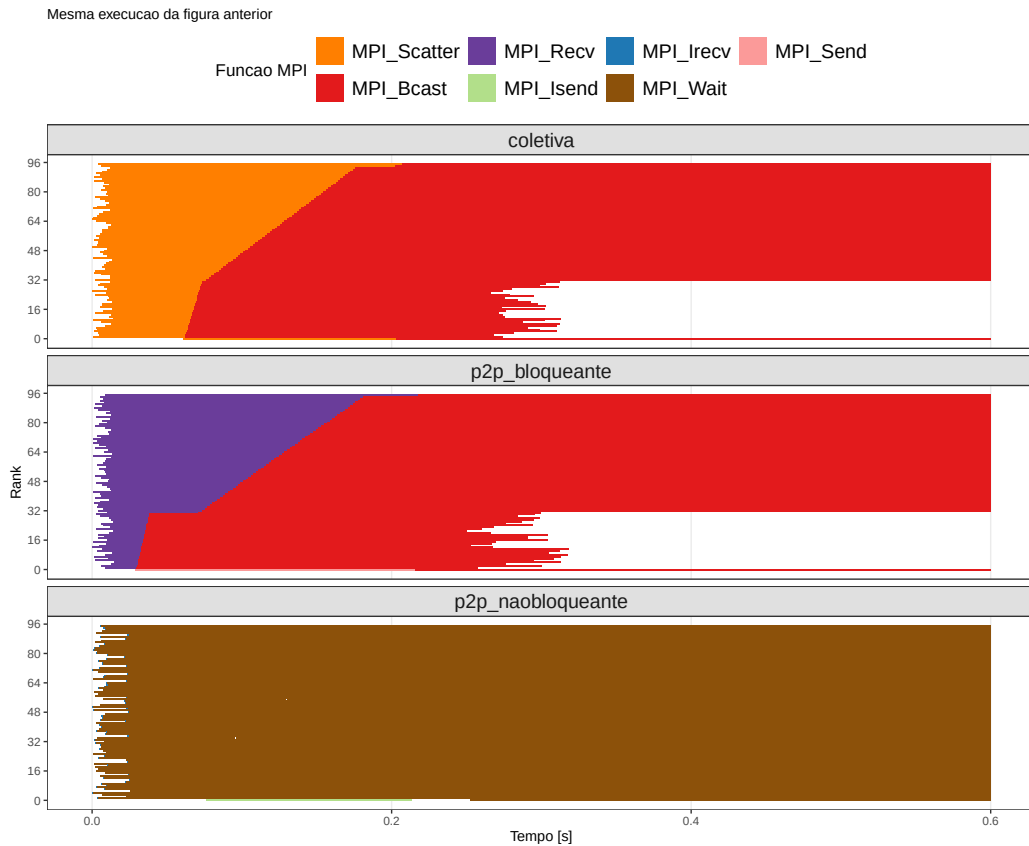


Figura 2: Zoom nos primeiros 0.6s da mesma execucao.

- **Coletiva:** todos entram juntos no **Scatter** e depois no **Bcast**. Os processos 0–31 (mesmo nó do mestre) saem do **Bcast** em $\sim 0.3s$ e começam a computar; os remotos só saem em $\sim 0.7s$, porque receber B pela rede demora mais.
- **Bloqueante:** os trabalhadores ficam parados no **Recv** esperando sua vez enquanto o mestre envia os 95 pedaços um a um, daí a borda roxa diagonal: quanto maior o rank, mais tarde o dado chega.
- **Não bloqueante:** os **Irecv** retornam na hora e a espera real fica no **Wait**. O rastreador não registra **MPI_Ibcast** como estado separado, então a espera pela matriz B também cai dentro desse **Wait**.

Tempo por operação MPI

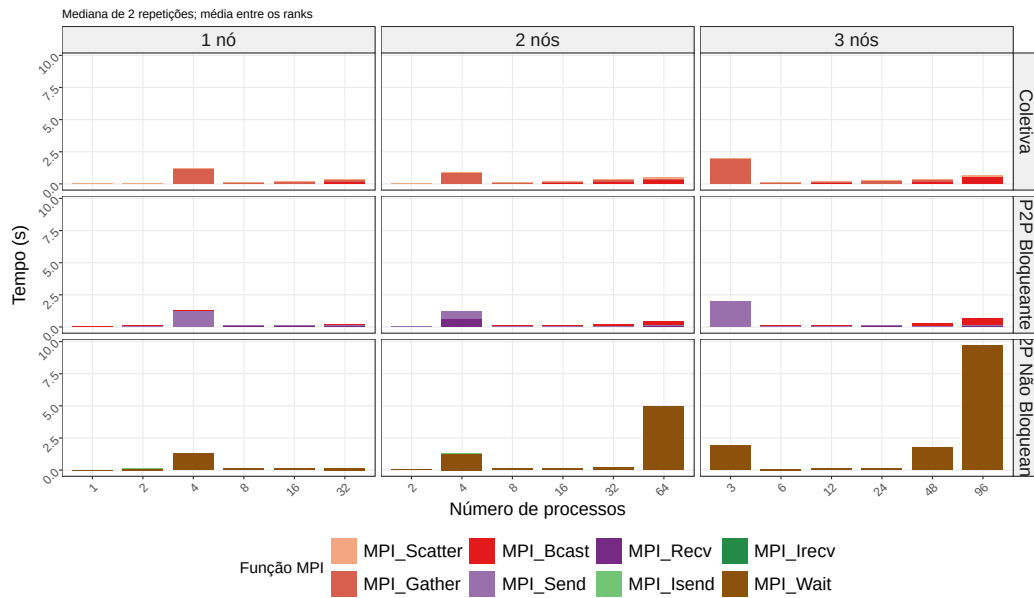


Figura 3: Tempo médio por processo em cada função MPI (size=1500, mediana de 2 repetições). Barras empilhadas por função.

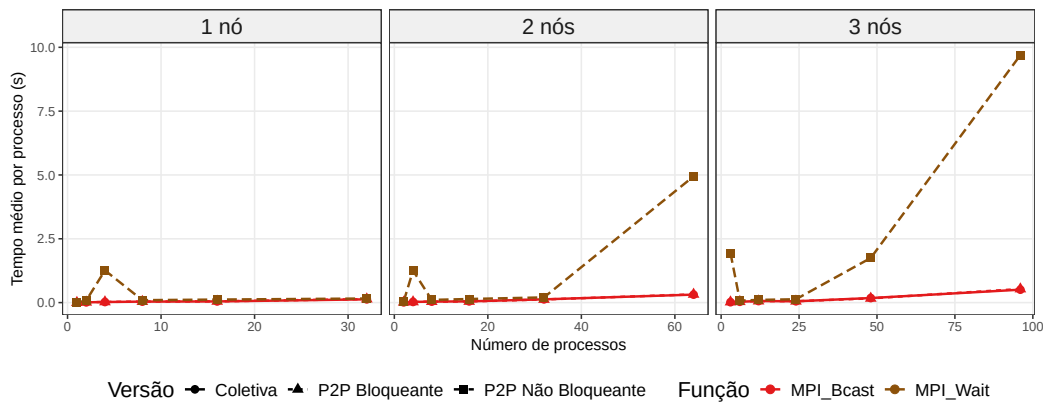


Figura 4: MPI_Wait (não bloqueante) vs MPI_Bcast (coletiva e bloqueante) para size=1500.

Tempo médio por processo (mediana de 2 execuções), size=1500, 3 nós:

Função	Versão	48 proc	96 proc
<code>MPI_Wait</code>	não bloqueante	1.75 s	9.70 s
<code>MPI_Bcast</code>	coletiva	0.18 s	0.51 s
<code>MPI_Bcast</code>	bloqueante	0.17 s	0.53 s
<code>MPI_Gather</code>	coletiva	0.11 s	0.07 s
<code>MPI_Scatter</code>	coletiva	0.08 s	0.10 s
<code>MPI_Recv</code>	bloqueante	0.06 s	0.11 s
<code>MPI_Send</code>	bloqueante	0.03 s	0.03 s
<code>MPI_Isend</code>	não bloqueante	0.002 s	0.002 s
<code>MPI_Irecv</code>	não bloqueante	0.0005 s	0.0005 s

Três coisas a notar:

1. Todas as operações custam décimos de segundo ou menos, exceto o `MPI_Wait` da versão não bloqueante: 1.75s com 48 processos e 9.7s com 96. Dobrar os processos multiplicou o custo por 5.5.
2. `Isend` e `Irecv` custam milésimos de segundo, mas isso é ilusão: eles só iniciam a operação. O custo real aparece no `Wait`.
3. `MPI_Bcast` custa praticamente o mesmo na coletiva e na bloqueante porque as duas chamam exatamente a mesma função da biblioteca.

Comparando execuções com o mesmo número de processos em 1, 2 ou 3 nós: `Scatter`, `Bcast` e `Recv` quase não mudam até ~32 processos, o que importa é quantos processos participam e quanto dado trafega, não quantos nós. Acima de 32 processos (só possível com 2 ou 3 nós) os custos coletivos crescem mais rápido e o `Wait` dispara.

Tabela completa: todas as combinações

Tempo médio por processo em cada função (segundos, mediana de 2 execuções). "Bcast (col)" e "Bcast (blq)" são o mesmo `MPI_Bcast` medido na versão coletiva e na bloqueante; "-" indica função não chamada; "0.000" significa menos de 1 milissegundo.

size	nos	nproc	Scatter	Gather	Bcast (col)	Send	Recv	Bcast (blq)	Isend	Irecv	Wait
500	1	1	0.002	0.001	0.000	-	-	0.000	-	-	0.000
500	1	2	0.005	0.002	0.001	0.001	0.004	0.002	0.001	0.000	0.007
500	1	4	0.007	0.010	0.005	0.001	0.007	0.005	0.001	0.000	0.012
500	1	8	0.005	0.002	0.004	0.000	0.010	0.005	0.000	0.000	0.011
500	1	16	0.014	0.014	0.008	0.000	0.008	0.009	0.000	0.000	0.015
500	1	32	0.014	0.008	0.050	0.001	0.012	0.035	0.001	0.000	0.043
500	2	2	0.005	0.004	0.001	0.001	0.005	0.002	0.001	0.000	0.006
500	2	4	0.007	0.004	0.004	0.001	0.008	0.005	0.001	0.000	0.009
500	2	8	0.008	0.007	0.007	0.001	0.011	0.004	0.000	0.000	0.013
500	2	16	0.011	0.011	0.007	0.001	0.015	0.008	0.000	0.000	0.022
500	2	32	0.018	0.009	0.029	0.001	0.031	0.015	0.000	0.000	0.022
500	2	64	0.057	0.005	0.072	0.001	0.035	0.045	0.001	0.001	0.488
500	3	3	0.006	0.004	0.002	0.001	0.007	0.003	0.001	0.000	0.008
500	3	6	0.008	0.026	0.006	0.001	0.010	0.007	0.000	0.000	0.011
500	3	12	0.010	0.021	0.007	0.000	0.010	0.006	0.000	0.000	0.016
500	3	24	0.007	0.006	0.011	0.001	0.013	0.012	0.000	0.000	0.021
500	3	48	0.038	0.004	0.057	0.002	0.039	0.057	0.002	0.000	0.251
500	3	96	0.044	0.004	0.125	0.001	0.033	0.097	0.001	0.001	1.080
1000	1	1	0.005	0.003	0.000	-	-	0.000	-	-	0.000
1000	1	2	0.015	0.008	0.005	0.003	0.023	0.008	0.002	0.000	0.016
1000	1	4	0.019	0.081	0.012	0.002	0.025	0.016	0.002	0.000	0.088

size	nos	nproc	Scatter	Gather	Bcast (col)	Send	Recv	Bcast (blq)	Isend	Irecv	Wait
1000	1	8	0.024	0.077	0.018	0.001	0.021	0.018	0.001	0.000	0.044
1000	1	16	0.019	0.072	0.022	0.001	0.037	0.023	0.001	0.000	0.056
1000	1	32	0.033	0.059	0.079	0.002	0.033	0.070	0.001	0.000	0.097
1000	2	2	0.015	0.011	0.005	0.002	0.014	0.007	0.002	0.000	0.016
1000	2	4	0.020	0.124	0.013	0.002	0.016	0.014	0.002	0.000	0.038
1000	2	8	0.019	0.081	0.019	0.001	0.034	0.019	0.001	0.000	0.031
1000	2	16	0.025	0.036	0.024	0.001	0.046	0.026	0.001	0.000	0.088
1000	2	32	0.031	0.047	0.105	0.001	0.039	0.036	0.001	0.000	0.069
1000	2	64	0.046	0.023	0.146	0.013	0.077	0.167	0.002	0.001	2.204
1000	3	3	0.018	0.011	0.008	0.002	0.016	0.010	0.002	0.000	0.022
1000	3	6	0.018	0.064	0.016	0.002	0.025	0.021	0.002	0.000	0.046
1000	3	12	0.023	0.049	0.032	0.002	0.031	0.037	0.001	0.000	0.051
1000	3	24	0.027	0.049	0.028	0.001	0.033	0.026	0.001	0.000	0.060
1000	3	48	0.045	0.029	0.092	0.016	0.064	0.119	0.001	0.000	0.793
1000	3	96	0.065	0.020	0.248	0.001	0.072	0.271	0.002	0.001	4.361
1500	1	1	0.010	0.006	0.000	-	-	0.000	-	-	0.000
1500	1	2	0.030	0.034	0.010	0.052	0.023	0.014	0.003	0.000	0.084
1500	1	4	0.051	1.167	0.023	1.196	0.043	0.026	0.003	0.000	1.261
1500	1	8	0.035	0.057	0.038	0.002	0.085	0.041	0.002	0.000	0.096
1500	1	16	0.050	0.097	0.046	0.002	0.082	0.048	0.002	0.000	0.119
1500	1	32	0.052	0.216	0.128	0.002	0.079	0.138	0.002	0.000	0.160
1500	2	2	0.030	0.019	0.010	0.024	0.022	0.014	0.003	0.000	0.047
1500	2	4	0.038	0.813	0.027	0.605	0.620	0.026	0.003	0.000	1.253
1500	2	8	0.034	0.046	0.039	0.002	0.066	0.040	0.002	0.000	0.105
1500	2	16	0.052	0.133	0.048	0.001	0.063	0.044	0.002	0.000	0.141
1500	2	32	0.066	0.202	0.127	0.002	0.059	0.122	0.003	0.000	0.203
1500	2	64	0.106	0.100	0.312	0.021	0.107	0.323	0.002	0.001	4.945
1500	3	3	0.036	1.936	0.017	1.948	0.031	0.022	0.003	0.000	1.939
1500	3	6	0.038	0.066	0.033	0.003	0.061	0.035	0.003	0.000	0.079
1500	3	12	0.051	0.103	0.061	0.002	0.053	0.059	0.002	0.000	0.105
1500	3	24	0.052	0.181	0.052	0.001	0.088	0.053	0.001	0.000	0.131
1500	3	48	0.076	0.113	0.178	0.025	0.060	0.166	0.002	0.000	1.748
1500	3	96	0.103	0.073	0.506	0.025	0.109	0.533	0.002	0.001	9.696

Coluna a coluna: **Scatter**, **Gather**, **Bcast**, **Send** e **Recv** ficam baratos em todas as combinações, raramente passando de 0.2s mesmo com 96 processos, o custo cresce suavemente com tamanho e número de processos, mas não explode. A coluna **Wait** é a única com comportamento explosivo: cresce com o tamanho (com 64 processos: 0.49s em size=500, 2.20s em 1000, 4.95s em 1500), cresce com os processos (em size=1500: 0.20s com 32, 1.75s com 48, 4.95s com 64, 9.70s com 96) e só aparece com mais de um nó, em 1 nó o **Wait** fica sempre abaixo de 0.16s.

Os valores altos isolados de **Gather**/**Send** em size=1500 com 3–4 processos (~1.2–1.9s) não são custo de comunicação: o mestre fica esperando dentro da chamada de coleta porque alguns trabalhadores terminam a computação mais devagar (desbalanceamento de memória).

Tempo total de execução

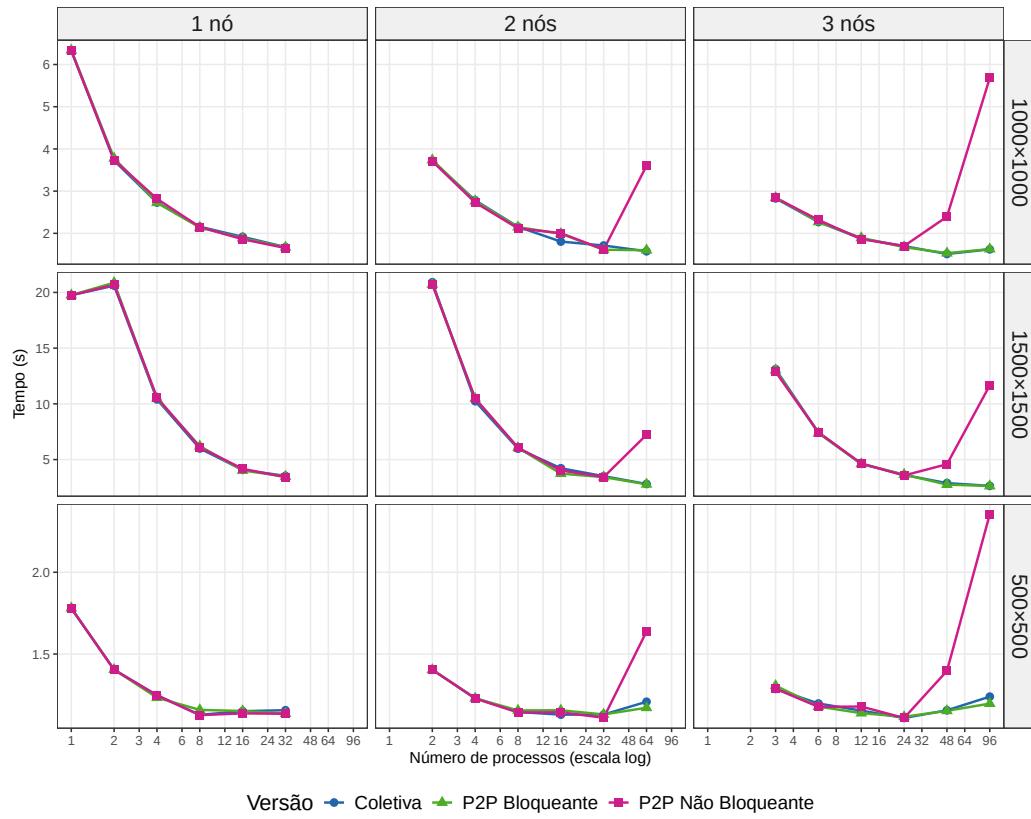


Figura 5: Tempo total de execução (média de 2 repetições) para todas as combinações experimentais. Eixo x em escala log.

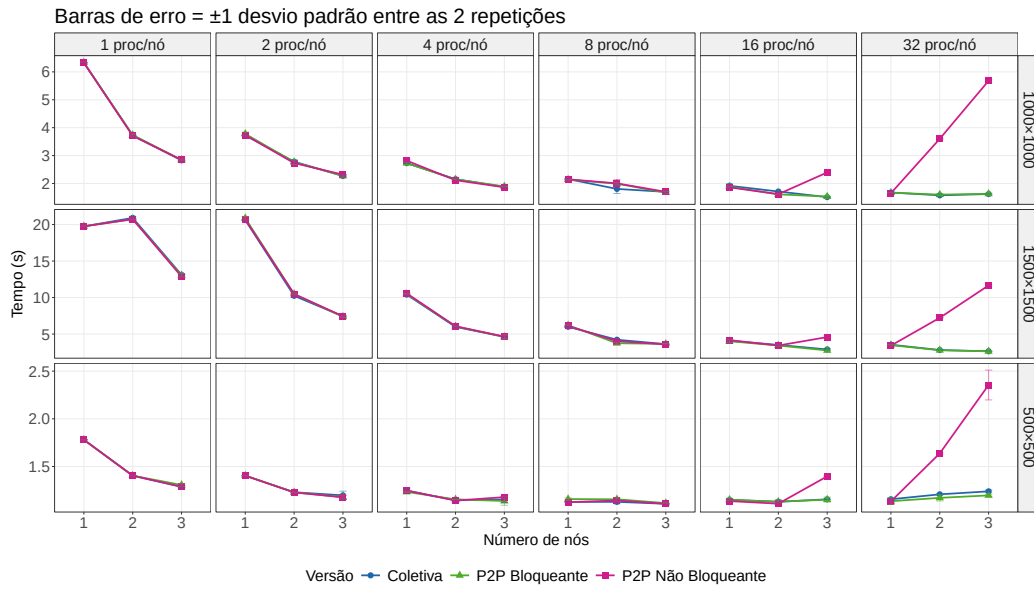


Figura 6: Escalabilidade por número de nós com número de processos por nó fixo. Barras de erro = ± 1 desvio padrão.

Nós	nproc	coletiva	p2p bloq.	p2p não bloq.
1	1	18.71 s	18.72 s	18.70 s
1	2	19.57 s	19.83 s	19.64 s
1	8	4.95 s	5.15 s	5.06 s
1	32	2.48 s	2.41 s	2.36 s
2	32	2.44 s	2.37 s	2.37 s
2	64	1.74 s	1.72 s	6.16 s
3	48	1.83 s	1.71 s	3.53 s
3	96	1.57 s	1.58 s	10.60 s

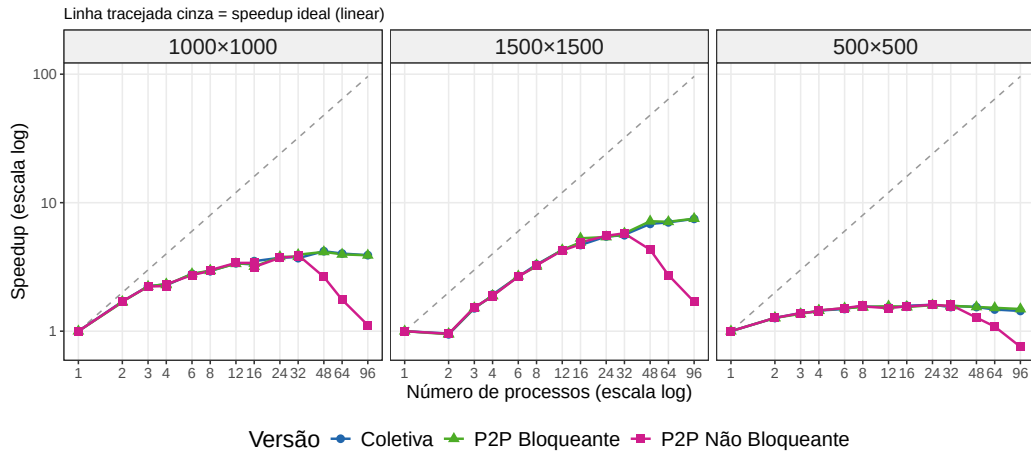


Figura 7: Speedup em relação à execução com 1 processo. Escala $\log \times \log$. Linha tracejada = speedup ideal.

Em um nó, as três versões empatam em todas as configurações: as diferenças ficam dentro da variabilidade entre repetições. A multiplicação em si leva segundos, enquanto a comunicação leva décimos de segundo.

Com 2 e 3 nós, coletiva e bloqueante continuam melhorando (o melhor tempo é 1.57s, com 96 processos), mas a não bloqueante piora: 6.16s com 64 processos e 10.60s com 96, quase 7x mais lenta que as outras.

Colapso da versão não bloqueante

Por que a versão assíncrona fica 7x mais lenta na maior configuração? O rastro permite responder passo a passo.

Cada trabalhador chama `MPI_Wait` exatamente 3 vezes: (1) esperar as linhas de A, (2) esperar a matriz B do `Ibcast`, (3) esperar o envio do resultado. As durações dos 3 `Wait` em uma execução com 96 processos:

Espera	Duração
(1) linhas de A (<code>Irecv</code>)	0.07 s
(2) matriz B (<code>Ibcast</code>)	9.1 s
(3) resultado (<code>Isend</code>)	~0 s

Praticamente todo o tempo é espera pela matriz B. O mesmo `MPI_Bcast` bloqueante custa 0.5s (tabela anterior), a diferença está na implementação: o `Bcast` bloqueante usa um algoritmo em árvore ($O(\log P)$ etapas), enquanto o `Ibcast` não bloqueante usa uma estratégia bem menos eficiente. Uma estimativa simples confirma: enviar B (18MB) para cada um dos 64 processos remotos individualmente por uma rede de 1Gb/s levaria $18\text{MB} * 64 / 125\text{MB/s} \sim 9.8\text{s}$, exatamente o que medimos. Em 1 nó não há rede, por isso o problema só aparece em execuções multi-nó.

Há dois fatores combinados: trocar chamadas bloqueantes por não bloqueantes sem computar nada entre o início e o `Wait` apenas move a espera de lugar, o algoritmo não sobrepõe computação e comunicação; e a implementação não bloqueante das coletivas pode ser bem pior que a bloqueante, diferença que só aparece quando a comunicação cruza a rede.

Conclusão

Em 1 nó, o padrão de comunicação é indiferente: a computação domina e as três versões empatam. Em múltiplos nós, coletiva e bloqueante escalam bem; a não bloqueante colapsa por causa do `MPI_Ibcast`, que não usa o algoritmo em árvore do `Bcast` bloqueante.

Comunicação assíncrona só compensa quando o algoritmo computa enquanto espera. Sem essa sobreposição, no melhor caso empata com as outras versões e no pior perde por detalhes de implementação da biblioteca.

Para este algoritmo, operações coletivas bloqueantes são a escolha mais sensata: desempenho igual ou melhor em todos os cenários, código mais simples e escalabilidade previsível.

Referências

Os experimentos utilizaram os recursos da infraestrutura PCAD, <http://pcad.inf.ufrgs.br>, no INF/UFRGS.